



Programa educativo:

**Licenciatura en Ingeniería en Tecnologías de la
Información e Innovación Digital**

Materia:

Estructura de datos

Unidad III:

Docente:

Gabriel barron

Grupo:

GTID141

Participantes:

Luis Angel valencia jantes

Fecha y Lugar de Entrega:

02 de diciembre del 2025, Dolores Hidalgo C.I.N., Gto.



Renaissance
nearpod

Name

Luis Angel Valencia Jantes

Nombre (optional)

Luis valencia

Únete a la lección →

Diccionario

Un **diccionario** es una estructura de datos que almacena información en pares clave-valor.

En Java, su equivalente directo es la interfaz `Map<K,V>`, donde:

- `K` = clave (Key)
- `V` = valor (Value)

Cada clave está asociada a un único valor, y no pueden existir claves duplicadas.

Ejemplo conceptual:

```
"ID" → 1284  
"Nombre" → "María"  
"Edad" → 22
```

Esto lo vuelve ideal para búsquedas rápidas, asociaciones, tablas, índices y catálogos.

Características

- ✓ Almacena pares *clave-valor*
- ✓ Las claves no se pueden repetir
- ✓ Los valores pueden repetirse
- ✓ La búsqueda se realiza por clave
- ✓ No forma parte de la jerarquía `Collection`, sino paralela
- ✓ Las implementaciones tienen distintos comportamientos internos

Componentes Principales

3.1 Clave (Key)

- No se puede duplicar
- Debe implementar correctamente `equals()` y `hashCode()` (para `HashMap`)
- Puede ser objeto, entero, string, etc.
- No es recomendable que sea mutable

3.2 Valor (Value)

- Puede repetirse
- Puede ser de cualquier tipo

3.3 Entrada (Entry)

Cada par clave-valor se encapsula en un `Map.Entry<K,V>`

Es usado para iterar:

```
for (Map.Entry<String, Integer> e : mapa.entrySet()) {  
    System.out.println(e.getKey() + " -> " + e.getValue());  
}
```

HashMap

- ✓ La más rápida en promedio
- ✓ Búsqueda $O(1)$
- ✓ No garantiza orden
- ✓ Permite **una clave null**
- ✓ Permite valores null
- ✓ No es sincronizada

Uso típico:

- índices
- cache
- acceso rápido

TreeMap

- ✓ Ordena automáticamente → orden natural o Comparator
- ✓ Implementa **NavigableMap**
- ✓ No permite claves null
- ✓ Operaciones $O(\log n)$
- ✓ Internamente usa un árbol rojo-negro

Ideal para:

- diccionarios ordenados
- autocompletar
- estructuras jerárquicas

LinkedHashMap

- ✓ Mantiene orden de inserción
- ✓ Similar a HashMap pero con LinkedList interna
- ✓ Útil para históricos o registros ordenados
- ✓ Se puede configurar para orden por "acceso" (LRU Cache)

HashTable

- ✓ Sincronizada
- ✓ NO permite null (ni clave ni valor)
- ✓ Obsoleta para la mayoría de los usos modernos
- ✓ Su equivalente moderno es `ConcurrentHashMap`

Operaciones Diccionarios

Operación	Método en Map	Descripción
Insertar	<code>put(k,v)</code>	Agrega o reemplaza
Insertar sin reemplazar	<code>putIfAbsent(k,v)</code>	No reemplaza si ya existe
Obtener valor	<code>get(k)</code>	Devuelve valor o null
Eliminar	<code>remove(k)</code>	Elimina por clave
Tamaño	<code>size()</code>	Número de pares
Verificar clave	<code>containsKey(k)</code>	True/False
Verificar valor	<code>containsValue(v)</code>	True/False
Limpiar	<code>clear()</code>	Vacía el map
Iterar	<code>entrySet()</code>	Iteración completa

Dicionarios

ACERTASTE 7 DE 7



100 CORRECTA
0 INCORRECTA
0 SIN RESPUESTA



LUIS ANGEL • Personal
jantesluis35@gmail.com

- Contraseñas y autorrellenar
- Configuración del perfil
- La sincronización está activada

Otros perfiles

- Perfil 1
- Configurar un perfil nuevo
- Explorar como invitado

¡CONSEGUIDO!

¡ENHORABUENA!

PUNTOS **61.2!**

 **Reintentar**

LUIS ANGEL • Personal
jantesluis35@gmail.com

- Contraseñas y autorrellenar
- Configuración del perfil
- La sincronización está activada

Otros perfiles

- Perfil 1
- Configurar un perfil nuevo
- Explorar como invitado